

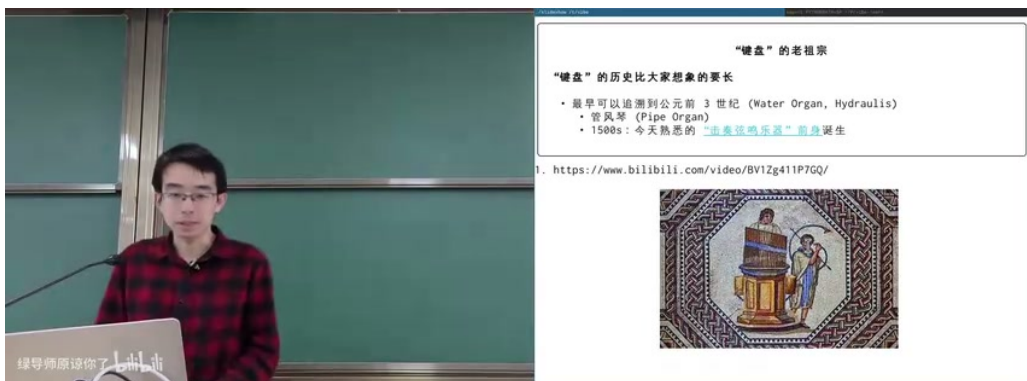
课程笔记

08 (Aside) - 终端和 UNIX Shell

2026 南京大学操作系统原理

lecture-to-notes

2026-06-20



视频作者/频道：南京大学操作系统原理

发布日期：未提供

视频时长：01:35:09

视频链接：未填写

目录

1 从键盘到终端：字符设备的来历	3
1.1 键盘先于计算机	3
1.2 从打字机到电传打字机	4
1.3 Video Teletypewriter 与 TTY	4
1.4 Unicode 扩展了字符终端的想象力	5
1.5 本章小结	6
2 终端作为输入输出设备	6
2.1 普通按键、修饰键与 escape code	6
2.2 Raw mode: 直接读终端字节	6
2.3 伪终端 PTY: 想要多少就能分配多少	7
2.4 Sixel: 字符流也能传图像	8
2.5 本章小结	8
3 终端驱动、行编辑与登录会话	9
3.1 Canonical mode: 为什么 cat 不逐字返回	9
3.2 系统启动、getty、login 与 controlling terminal	9
3.3 Textual: 字符终端也能有复杂界面	10
3.4 本章小结	10
4 Ctrl-C、信号与 Job Control	11
4.1 Ctrl-C 到底杀谁	11
4.2 Signal 只是机制，目标集合才是难点	12
4.3 Session、process group 与前台进程组	12
4.4 Job control 像字符版窗口管理器	13
4.5 历史遗留与现代替代	14
4.6 本章小结	14
5 UNIX Shell：一门面向系统调用的语言	14
5.1 最小 Shell 可以很小	14
5.2 Shell 是文本替换语言	15
5.3 重定向与管道是文件描述符操作	15
5.4 为什么还要读 man sh	16
5.5 Shell 的缺点与无奈	16
5.6 本章小结	17
6 总结与延伸	17
6.1 核心机制回顾	17

6.2	从历史包袱到设计启发	17
6.3	下一步可以做什么	17
6.4	本章小结	18

1 从键盘到终端：字符设备的来历

这节课不是从系统调用或 Shell 语法开始，而是从一个更老的问题开始：为什么我们今天还能用一个只能输入和输出字符的窗口完成几乎所有开发工作？答案藏在键盘、打字机、电传打字机、视频终端和 UNIX 的连续演化中。终端不是突然出现的抽象接口，而是打字机式人机交互在计算机时代的延续。

1.1 键盘先于计算机

键盘的历史比计算机长得多。水风琴、管风琴、早期钢琴都使用按键把人的手指动作转换成机械动作；到了打字机，按键不再击弦发声，而是推动字模击打色带，在纸上留下字符。这个机制留下了今天终端世界中的一些“化石”：

- **Shift/Caps Lock**：来自机械打字机中字模档位上移和锁定的设计。
- **Carriage Return, \r**：打印头回到行首。
- **Line Feed, \n**：纸向前走一行。
- **Space/Backspace**：一个右移，一个左移；在纸上打错字时，退格后用符号覆盖或划掉。
- **Tab**：跳到预设的 tab stop，用于制表和对齐。
- **等宽字体**：每次击打后机械位置移动固定距离，因此每个字符占同样宽度。



图 1：机械打字机把键盘、色带、字模、回车、退格和等宽排版这些概念连接起来，是今天终端行为的物理祖先。¹

¹视频画面时间区间：00:04:30–00:04:35。

终端继承的是“字符位置”模型

打字机把人机交互抽象成一个当前位置：输入普通字符就在当前位置留下字形；输入控制字符就移动当前位置、换行、回车或调整状态。早期终端继承的正是这个模型，而不是图形界面的像素模型。

1.2 从打字机到电传打字机

打字机只能在本地纸张上留下文字。电传打字机的跨时代之处在于：按下一个键，不是在本地立刻打印，而是把字符编码成电信号，在远端的机器上打印。只要能给字符编号，字符就能变成比特串，也就能被计算机处理。

早期电传系统使用 5-bit Baudot code。它的字符集合很小，但已经具有后来的终端思想：普通字符用于文本，特殊编码用于切换状态或控制设备。这也解释了为什么 ASCII 表前面一大段不是可显示字符，而是控制字符。



图 2: 电传打字机将字符映射为有限位宽编码；“字符表”从一开始就同时服务于文本内容和设备控制。²

1.3 Video Teletypewriter 与 TTY

计算机最早的输出可以是打孔纸带或二进制孔洞，但人类需要可读输出，也希望避免用穿孔卡片写程序。于是电传打字机成为计算机的输入输出设备。TTY 这个名字来自 *teletypewriter*：从操作系统看，它提供的基本接口近似于：

```
1 void putchar_to_terminal(char ch);  
2 char getchar_from_terminal(void);
```

Listing 1: 从操作系统视角看终端输出的最小接口

²视频画面时间区间：00:11:10–00:11:15。

把字符写给终端，它就显示；从终端读字符，它就把用户按键转换成字节流交给系统。VT100 是 1978 年的里程碑产品，奠定了 80x24 字符、ANSI escape sequence、屏幕滚动、光标控制和许多事实标准。

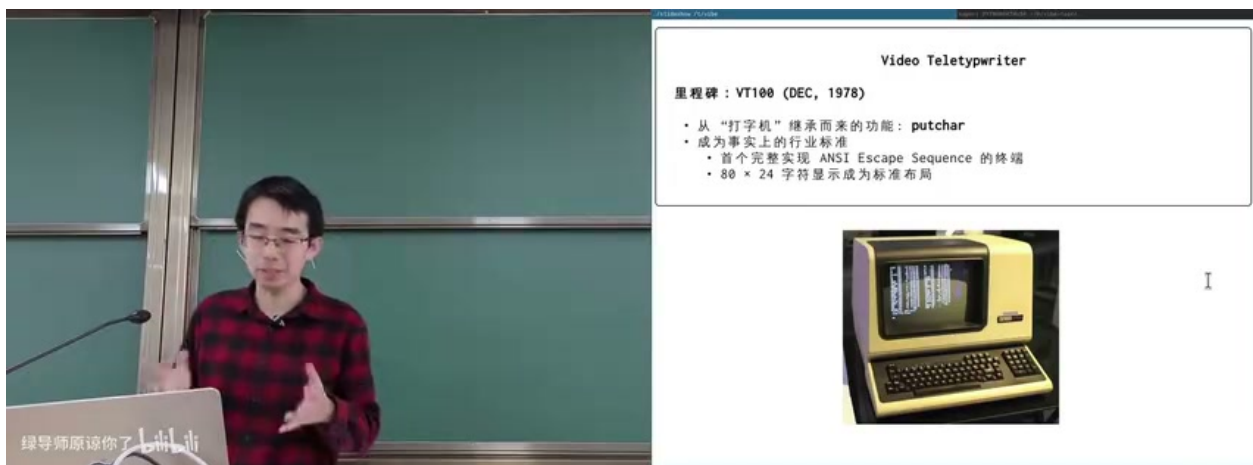


图 3: VT100 把视频终端变成事实标准：字符显示、光标控制、escape sequence 和 80x24 布局都延续到今天。³

1.4 Unicode 扩展了字符终端的想象力

终端原本只能显示有限字符，但 Unicode 让字符空间大幅扩张：方块、线条、数学符号、表情符号、组合字符都可以进入终端。于是“字符终端”不再只像打字机，也能绘制状态栏、进度条、边框、表格甚至图形化风格的界面。

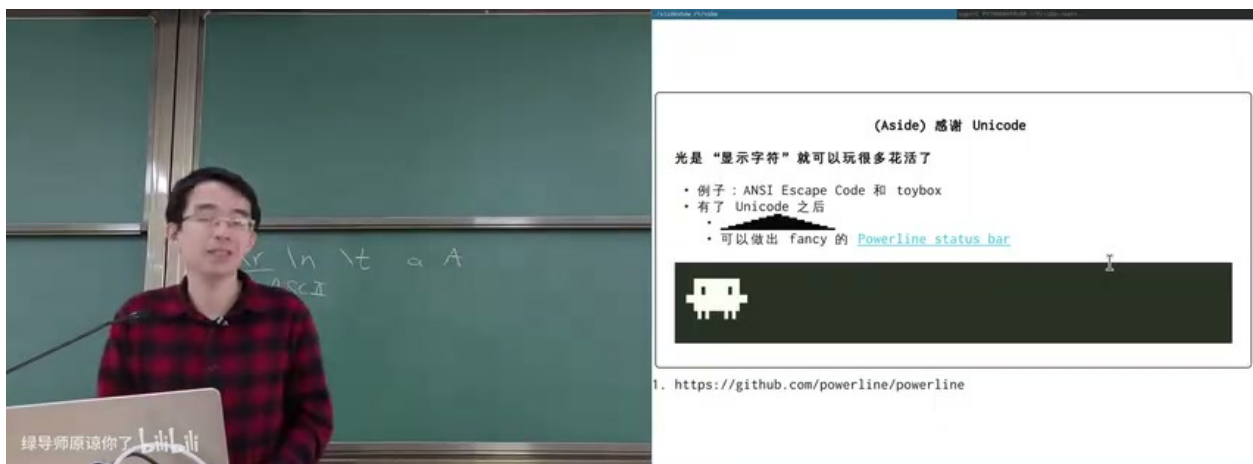


图 4: Unicode 让终端可以用字符拼出图形元素；Powerline 风格状态栏和字符画都建立在这一点上。⁴

³视频画面时间区间：00:14:05–00:14:10。

⁴视频画面时间区间：00:18:40–00:18:45。

1.5 本章小结

终端的核心不是“黑窗口”，而是字符流设备。打字机给出了当前位置、回车、退格、Tab、等宽字体；电传打字机把字符编码成可远程传输的信号；VT100 把字符显示和屏幕控制标准化；Unicode 又把字符终端扩展成可表达复杂界面的媒介。理解这条历史线，后面关于 Ctrl-C、PTY、Shell 的设计就不再显得突兀。

2 终端作为输入输出设备

2.1 普通按键、修饰键与 escape code

终端不是键盘硬件本身，而是把用户输入转换成字符流的设备。按下 A，终端发送字符 A；按下 Shift 本身，终端通常不发送字符；按下 Shift+A，终端发送大写 A。Ctrl 键则更有历史感：Ctrl+C 会产生 ASCII 码 03，Ctrl+D 会产生 ASCII 码 04。

问题在于，ASCII 表中没有“方向键”“颜色”“清屏”“鼠标点击”等概念。解决方法是 escape sequence：先发送 ESC 字符，再发送一串约定好的后续字符。例如方向键常被编码为以 ESC 开头的序列，颜色和光标控制也通过类似机制完成。

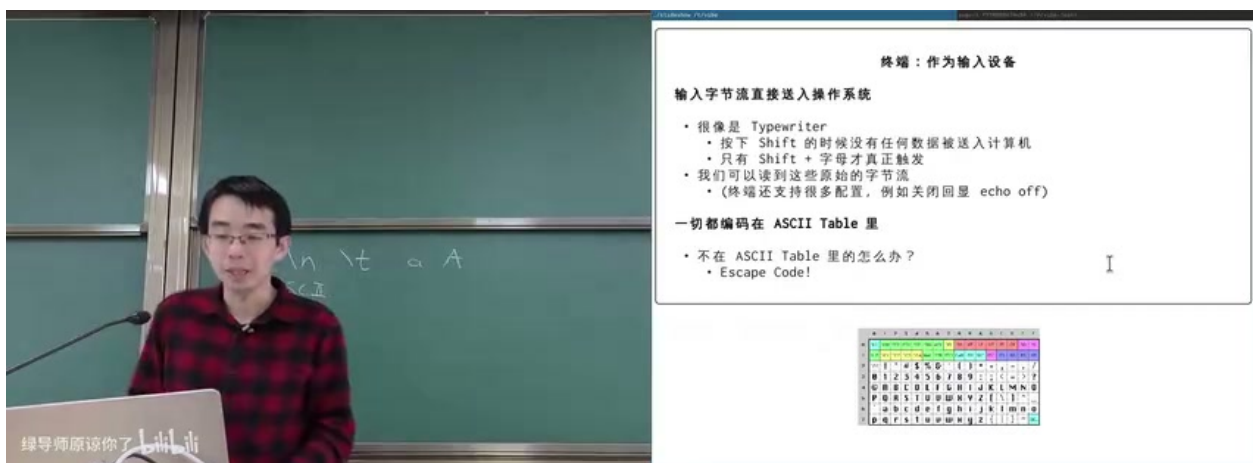


图 5: 终端输入仍然以 ASCII/escape sequence 为基础；普通字符、控制字符和无法直接编码的特殊键共同构成输入字节流。⁵

2.2 Raw mode: 直接读终端字节

如果程序把终端配置成 raw mode，就可以绕过默认的行编辑，把终端送来的字节几乎原样交给应用。此时 Ctrl-C 不再一定终止程序，它只是一个字节 0x03；Ctrl-Z 也不一定挂起程序，它只是另一个控制字符。课堂中的演示程序保留了 q 作为退出后门，否则需要从另一个终端杀掉进程。

⁵ 视频画面时间区间：00:23:15–00:23:20。

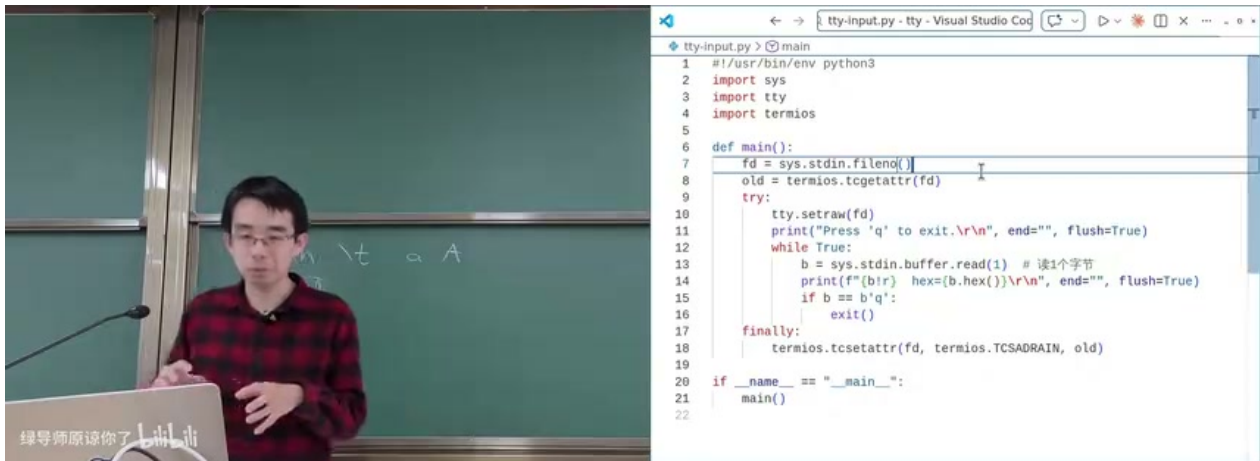


图 6: raw mode 演示程序直接读取 `stdin` 字节并打印十六进制值，说明 `Ctrl-C` 首先是一个输入字符。⁶

Ctrl-C 不是魔法按键

`Ctrl-C` 的默认效果不是键盘硬件直接杀死进程，而是终端驱动在特定配置下把字符解释成中断请求，再由内核向目标进程组发送信号。关闭这层解释后，它就只是一个普通控制字符。

2.3 伪终端 PTY：想要多少就能分配多少

今天的“终端”通常是终端模拟器加伪终端。真实 VT100 是一台物理设备；伪终端则由操作系统动态分配，应用程序看到的是 `/dev/pts/N` 这样的设备文件。伪终端通常成对出现：

- **master 端**：由终端模拟器、`tmux`、`ssh`、`ttymrec` 等程序持有。
- **slave 端**：交给 `shell` 或被启动的交互程序，它以为自己连接在一个真实终端上。

典型分配流程是打开 `/dev/ptmx`，执行 `grantpt`、`unlockpt`，再得到 `slave` 路径。这样，`tmux` 可以在一个外层终端中创建多个内层终端，`ssh` 可以给远程登录分配伪终端，`ttymrec` 可以夹在 `master/slave` 之间记录完整输入输出。

⁶视频画面时间区间：00:27:30–00:27:35。



图 7: 课堂中的 miniTTY 演示: 外层终端是 `/dev/pts/3`, 内层伪终端是 `/dev/pts/4`, 两者通过 master/slave 数据转发连接。⁷

2.4 Sixel: 字符流也能传图像

终端接口看起来只交换字节流, 但字节流可以携带约定好的图像协议。Sixel 的基本思想是用可显示字符编码 6 像素高的位图块, 再用调色板和重复机制扩展颜色与压缩。现代终端如 Kitty、foot、Windows Terminal 的新版本都支持或部分支持图像协议。

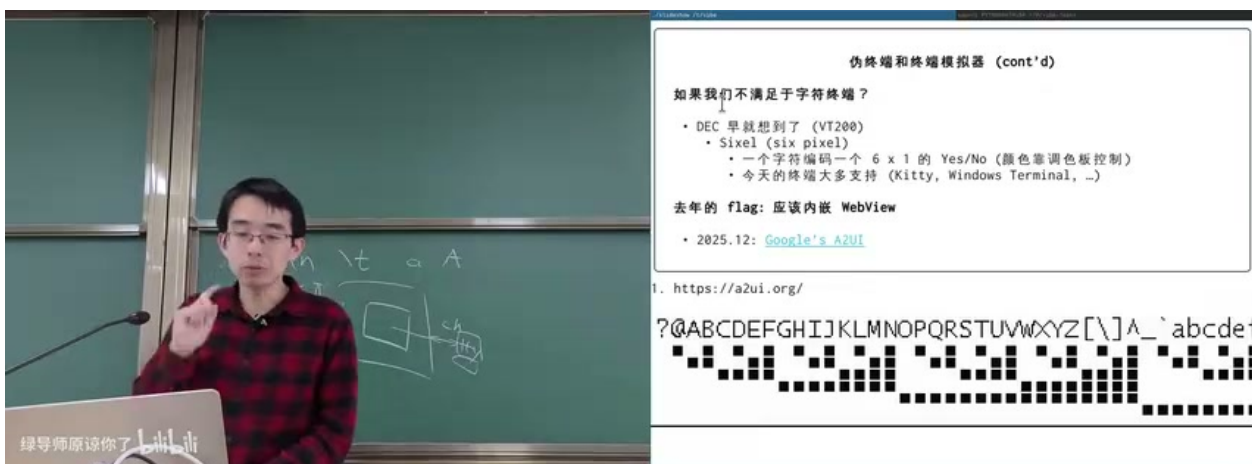


图 8: Sixel 把图像拆成 6 像素高的块并编码成字符序列; 终端仍然接收字节流, 但渲染结果可以是位图。⁸

2.5 本章小结

终端和应用程序之间的基本契约是字节流。普通字符、Ctrl 组合键、方向键、颜色、光标移动、图像协议都可以放进这条字节流中。所谓“终端很强”, 不是因为接口复杂, 而是因为几十年间人们在简单接口上叠加了大量兼容约定。

⁷ 视频画面时间区间: 00:28:15–00:28:20。

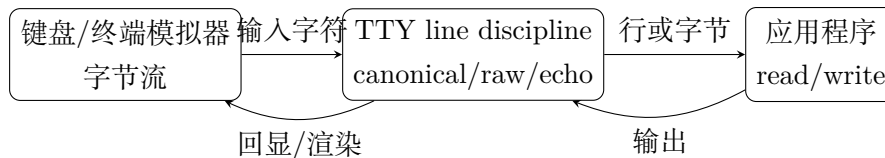
⁸ 视频画面时间区间: 00:36:10–00:36:15。

3 终端驱动、行编辑与登录会话

3.1 Canonical mode: 为什么 cat 不逐字返回

如果终端只是逐字节输入输出，运行 cat 时每按一个键，read 就应该返回。但实际情况是：你可以先输入一整行，退格修改，直到按回车后 read 才返回。这是 canonical mode 的效果。终端驱动在内核中维护一个行缓冲区，并负责：

- 收集字符直到行结束；
- 处理 Backspace、Ctrl-U 等行编辑字符；
- 按配置决定是否 echo；
- 把 Ctrl-C、Ctrl-D 等特殊字符解释成信号或 EOF。



canonical mode 的本质

canonical mode 不是终端模拟器给 Shell 加的高级功能，而是 TTY 驱动提供的行规程：应用看到的是已经经过行编辑和特殊字符解释的输入，而不是原始按键流。

3.2 系统启动、getty、login 与 controlling terminal

系统启动后，第一个用户态进程并不天然拥有标准输入、输出、错误。传统 UNIX/Linux 会由 init 在各个终端上启动 getty。getty 打开指定 tty 设备，设置波特率等参数，把文件描述符 0、1、2 指向该终端，然后启动登录程序。

远程登录类似但多一层 PTY：sshd 为连接分配伪终端，并把登录 shell 连接到这个 slave 端。用户由此获得一个 session 和一个 controlling terminal。

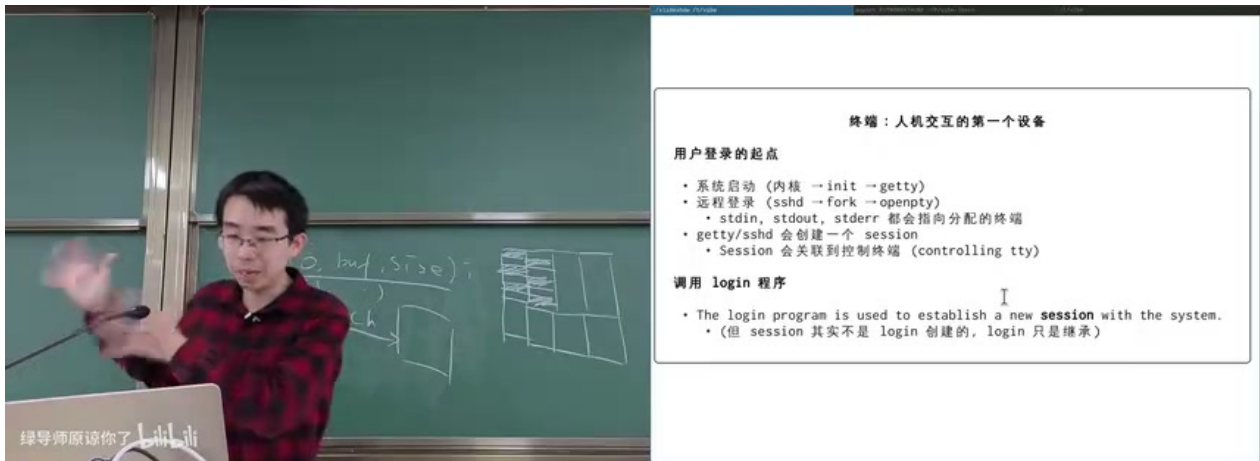


图 9: 登录的起点：本地 `getty` 或远程 `sshd` 分配终端，设置标准文件描述符，并建立用户会话与 controlling terminal 的联系。⁹

3.3 Textual: 字符终端也能有复杂界面

一旦终端支持 Unicode、颜色、鼠标事件和 escape sequence，就可以构造接近图形界面的 TUI。课堂展示的 Python Textual 库可以在终端中绘制窗口、滚动条、表格、复选框、单选按钮、主题切换甚至小游戏。它仍然没有跳出终端模型：底层依旧是应用和终端之间交换字符与控制序列。

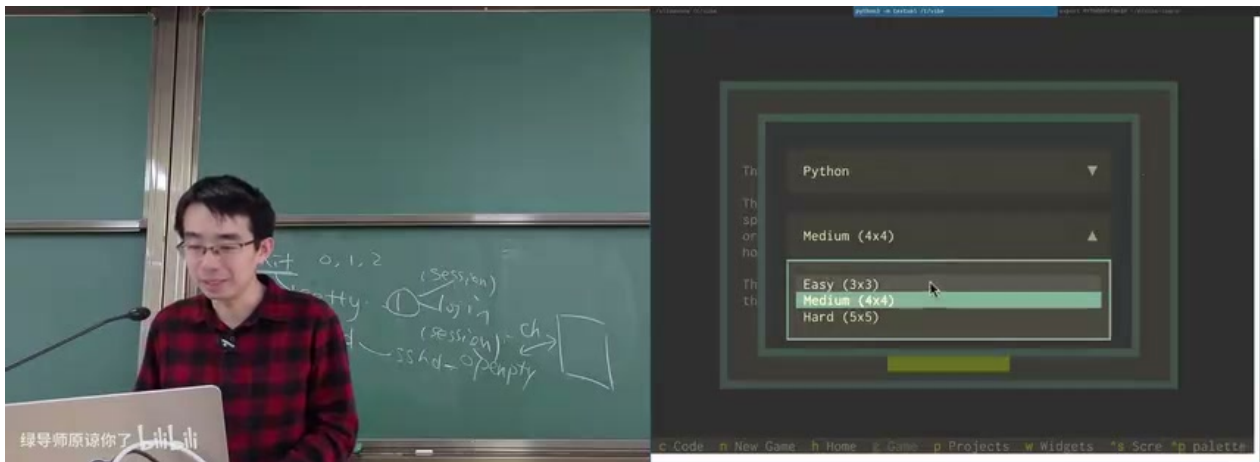


图 10: Textual TUI 在字符终端中渲染多层窗口和菜单，说明“字符流”可以承载非常丰富的人机交互。¹⁰

3.4 本章小结

终端驱动把简单字节流加工成符合用户直觉的交互：行编辑、回显、EOF、信号都在这层发生。登录过程把进程、文件描述符、终端设备和 session 绑定起来；终端模拟器和 TUI

⁹视频画面时间区间：00:42:25–00:42:30。

¹⁰视频画面时间区间：00:49:55–00:50:00。

库又在同一接口上构建出接近图形界面的体验。

4 Ctrl-C、信号与 Job Control

4.1 Ctrl-C 到底杀谁

只考虑一个进程时，Ctrl-C 的解释相对简单：终端驱动把字符解释成 interrupt，内核向进程发送 SIGINT，默认处理是终止。如果程序注册了信号处理函数，就可以覆盖默认行为。

```
1 void handler(int sig) {
2     if (sig == SIGINT) {
3         printf("received_SIGINT\n");
4         return;
5     }
6     if (sig == SIGQUIT) {
7         printf("received_SIGQUIT\n");
8         exit(0);
9     }
10 }
11
12 int main() {
13     signal(SIGINT, handler);
14     signal(SIGQUIT, handler);
15     while (1) pause();
16 }
```

Listing 2: 信号处理的核心想法：注册处理函数，异步跳转执行

真正麻烦的是多进程：一个前台命令可能 fork 出许多子进程；子进程的父进程可能提前退出，被 1 号进程接管；后台也可能有别的任务正在运行。按 Ctrl-C 时，我们希望杀掉“当前前台任务”的所有相关进程，而不是误伤 shell、编辑器、后台任务或别的终端。

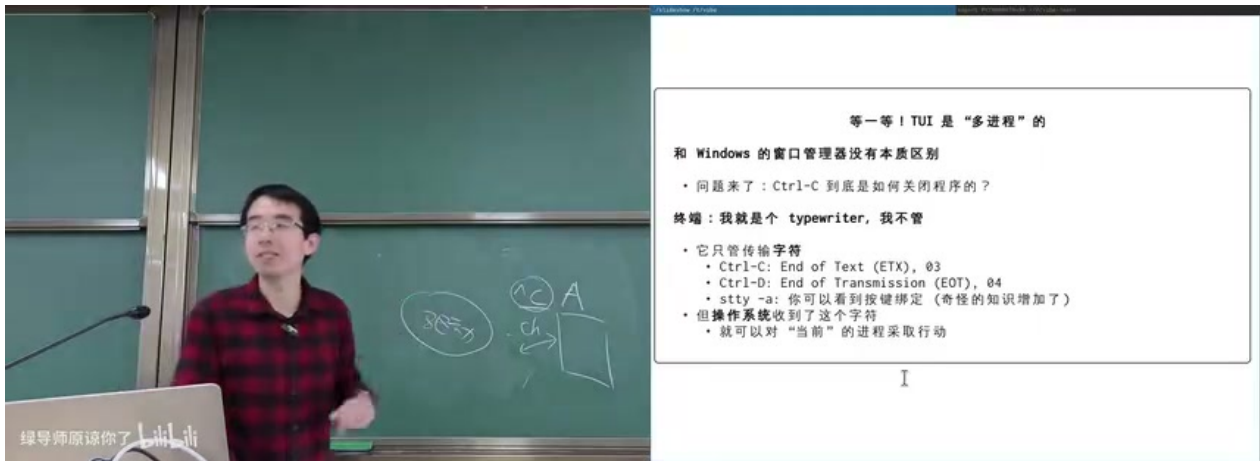


图 11: Ctrl-C 的目标不是“一个进程”，而是当前前台交互任务；这一问题迫使 UNIX 引入 session 和 process group。¹¹

4.2 Signal 只是机制，目标集合才是难点

信号机制回答“如何异步打断进程”，但没有回答“打断谁”。课堂中强调：如果前台程序派生了许多进程，Ctrl-C 应该让所有属于该前台任务的进程收到 SIGINT；如果后台任务正在运行，它不应被 Ctrl-C 打断。

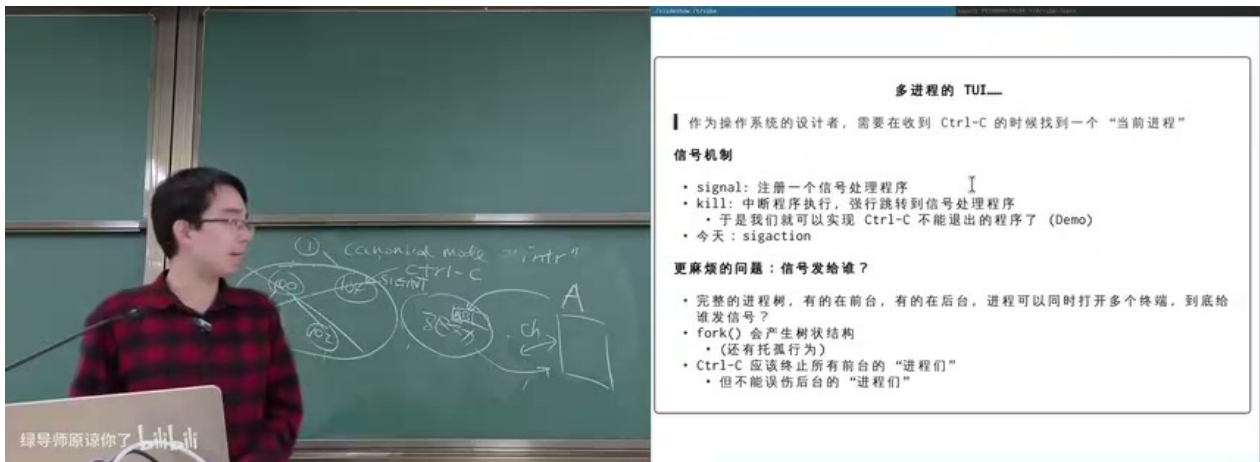


图 12: 多进程 TUI 的核心问题：作为操作系统设计者，必须知道 Ctrl-C 的目标是哪个“当前进程集合”。¹²

4.3 Session、process group 与前台进程组

UNIX 的解决方案是增加两层分组：

- **Session**：通常由一次登录或一个终端会话建立，关联一个 controlling terminal。

¹¹ 视频画面时间区间：00:52:50–00:52:55。

¹² 视频画面时间区间：01:05:00–01:05:05。

- **Process group**: session 内更小的分组，对应 shell job。一个 pipeline 或前台命令通常在同一个 process group 中。
- **Foreground process group**: 终端当前把特殊按键和作业控制信号发送给这个进程组。

因此 Ctrl-C 的精确定义变成：TTY 驱动在 canonical/signal 相关配置下，把 interrupt 字符解释成向前台进程组发送 SIGINT。Ctrl-Z 类似，会向前台进程组发送停止信号；shell 再用 fg/bg、tcsetpgrp 等机制切换前台进程组。

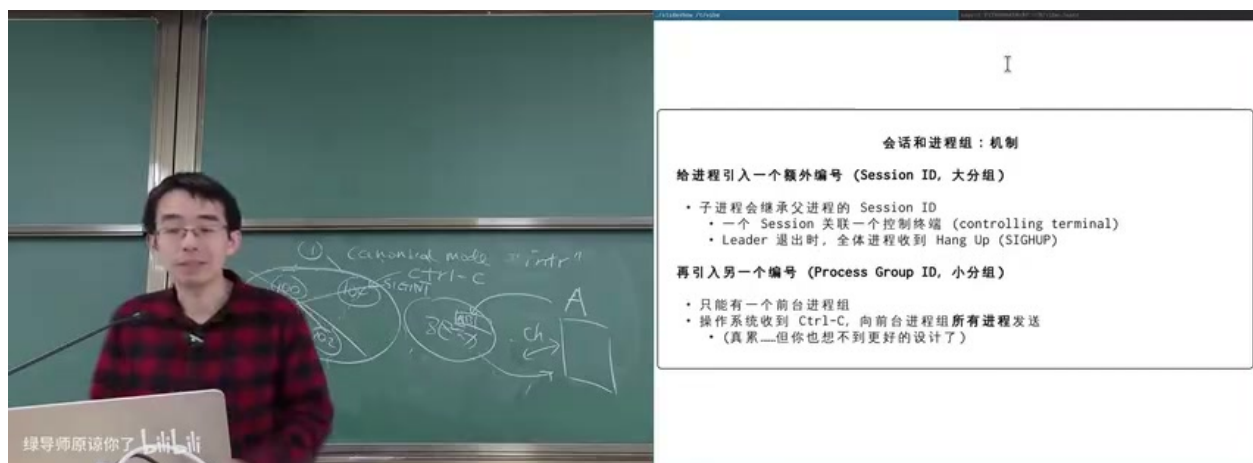


图 13: session 是大分组，process group 是 job control 的小分组；终端只把 Ctrl-C 等信号发给前台进程组。¹³

为什么 SSH 断开会让程序退出

session 关联 controlling terminal。当连接断开或 session leader 退出时，相关进程可能收到 SIGHUP，默认动作是退出。nohup、tmux、screen 等工具的价值之一，就是让长任务不直接依赖易断开的登录终端。

4.4 Job control 像字符版窗口管理器

在只有一个字符终端的时代，shell 仍然要提供类似“窗口切换”的体验。vim a 后按 Ctrl-Z，相当于把当前 job 暂停到后台；再运行 vim b，可以形成多个 job；jobs 查看列表；fg %1 把某个 job 放回前台；bg %3 让暂停的 job 在后台继续运行。

```

1 $ vim a.txt
2 ^Z
3 [1]+  Stopped                  vim a.txt
4 $ vim b.txt
5 ^Z
6 [2]+  Stopped                  vim b.txt
  
```

¹³视频画面时间区间：01:10:55–01:11:00。


```

7 $ jobs
8 [1]-  Stopped                  vim a.txt
9 [2]+  Stopped                  vim b.txt
10 $ fg %1

```

Listing 3: Shell job control 的典型交互

4.5 历史遗留与现代替代

讲者特别指出，session/process group/job control 是历史条件下的工程方案：它把“应用”“窗口”“终端设备”“进程集合”揉在一起，成为 POSIX 兼容性的一部分。今天如果重新设计系统，可以用更直接的隔离单元：Android 用 UID 隔离应用并按 UID 杀进程；容器、namespace、cgroup、虚拟机都能表达“启动一个任务集合，再整体终止”的概念。

4.6 本章小结

Ctrl-C 的完整链路是：键盘产生控制字符；TTY 驱动在当前终端配置下解释它；内核向前台 process group 发送信号；各进程按默认动作或自定义 handler 响应。理解这条链路，就能解释 raw mode 中 Ctrl-C 为什么失效、后台任务为什么不被杀、SSH 断开为什么会影响长任务，以及 tmux/nohup 为什么有用。

5 UNIX Shell：一门面向系统调用的语言

5.1 最小 Shell 可以很小

如果不实现 job control、复杂语法和交互式补全，一个 Shell 的骨架极其简单：

```

1 while (1) {
2     printf("$ ");
3     line = read_line(STDIN_FILENO);
4     argv = split_by_spaces(line);
5     pid = fork();
6     if (pid == 0) {
7         execve(argv[0], argv, environ);
8         _exit(127);
9     }
10    waitpid(pid, NULL, 0);
11 }

```

Listing 4: 最小 Shell 的同步骨架

Shell 的本质不是“命令输入框”，而是把一行文本翻译成一组系统调用：fork、execve、waitpid、open、dup2、pipe、setpgid、tcsetpgrp 等。

5.2 Shell 是文本替换语言

Shell 不是 C 语言那样的数据类型丰富的语言。早期 Shell 甚至没有内建整数；要计算 $1 + 2$ ，可以调用外部程序 `expr 1 + 2`。这体现了 UNIX philosophy：每个工具做一件事，Shell 负责把工具连接起来。

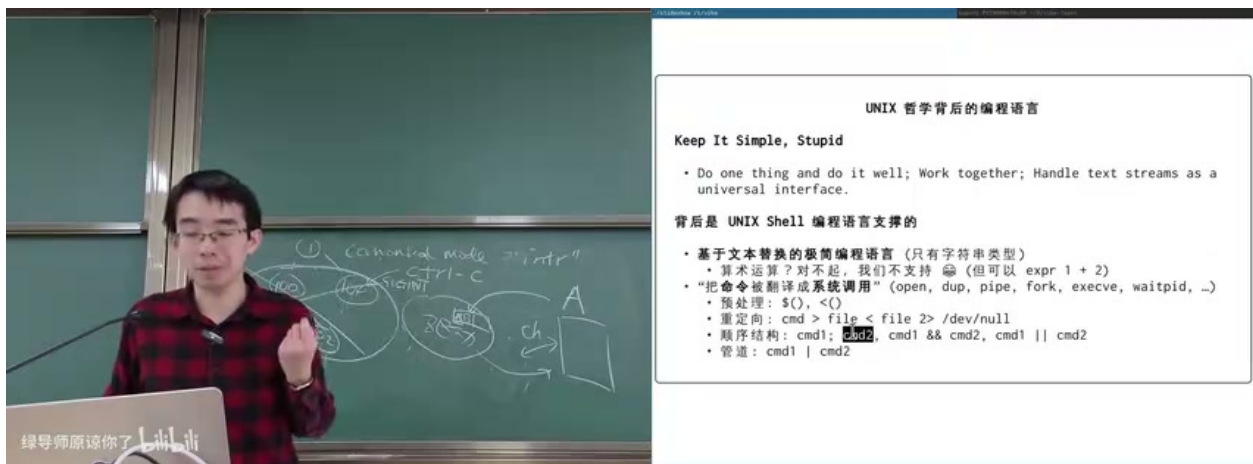


图 14: Shell 背后的编程语言服务于 UNIX philosophy：用文本流、进程、文件描述符和管道组合小工具。¹⁴

常见语义包括：

- **命令替换**：`$(cmd)` 把命令输出作为字符串粘贴进当前命令。
- **进程替换**：`<(cmd)` 把命令输出包装成类似文件的路径，如 `/dev/fd/63`。
- **重定向**：`cmd > file` 在子进程 `execve` 前把 `fd 1` 指向文件。
- **管道**：`cmd1 | cmd2` 创建 pipe，把左侧 `stdout` 接到右侧 `stdin`。
- **控制流**：`cmd1 && cmd2`、`cmd1 || cmd2` 使用退出状态做短路求值。

5.3 重定向与管道是文件描述符操作

重定向不是被执行程序的特殊能力，而是 Shell 在 `execve` 前布置好文件描述符。执行：

```
1 $ ./a.out > a.txt
```

Listing 5: 标准输出重定向

Shell 会 fork 子进程，在子进程中打开 `a.txt`，用 `dup2` 让 `fd 1` 指向这个文件，再执行 `./a.out`。程序本身仍然只是向 `stdout` 写数据。

管道也类似。Shell 调用 `pipe` 得到一读一写两个 `fd`，再 fork 左右两个子进程：左边关闭读端，把 `stdout` 指向写端；右边关闭写端，把 `stdin` 指向读端；最后分别 `execve`。

¹⁴视频画面时间区间：01:27:50–01:27:55。

Shell 语法的操作系统含义

Shell 的每个语法糖几乎都能翻译成文件描述符和进程管理：`>` 是 `fd 1` 的替换，`<` 是 `fd 0` 的替换，`|` 是 `pipe` 加两个 `dup2`，`&` 是不等待或放入后台 `job`，`fg/bg` 是前台进程组切换。

5.4 为什么还要读 `man sh`

讲者在 AI 时代仍然建议学生遍历 `man sh`。原因不是要背语法，而是扩展“什么能办到”的想象力。手册里藏着大量设计者已经想到的能力：特殊参数、here document、追加重定向、`stderr` 重定向、`case/for/while`、函数、变量展开、命令替换、算术扩展、`trap` 等。知道这些能力存在，才能更好地向 AI 提问、设计脚本、判断模型生成的命令是否合理。

5.5 Shell 的缺点与无奈

Shell 强大，但也有历史包袱。基于文本替换的语言很容易踩坑：空格决定分词，引用规则复杂，变量为空会改变参数个数，不同 Shell 对边界情况可能行为不同。课堂展示的例子包括重定向和管道组合在不同 Shell 中语义不一致，以及 `sudo echo hello > /etc/a.txt` 不能按直觉获得 `root` 写权限。

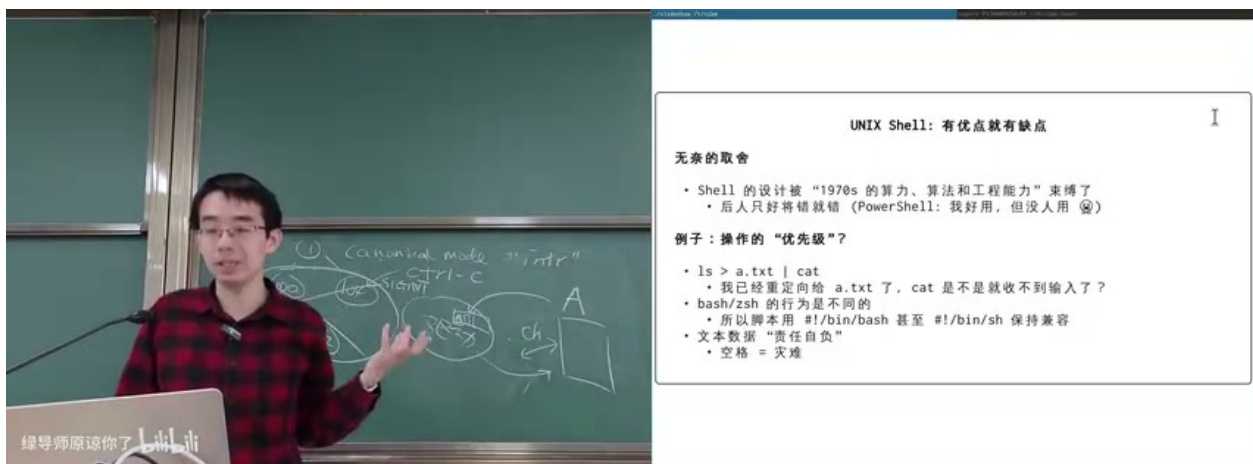


图 15: Shell 的文本替换和重定向语义会制造反直觉行为；理解 `fd` 归属比记住命令表面形式更重要。¹⁵

`sudo echo ... > file` 的陷阱

`sudo` 提升的是 `echo` 进程的权限；重定向 `> file` 是当前 Shell 在执行 `sudo` 前完成的。如果当前 Shell 没有写目标文件的权限，重定向会先失败。常见修法是使用 `sudo sh -c 'echo ... > file'` 或 `echo ... | sudo tee file`。

¹⁵ 视频画面时间区间：01:34:05–01:34:10。

5.6 本章小结

Shell 是一门极简但非常贴近操作系统的语言。它的强项是把进程、文本流、文件描述符和控制流粘合起来；它的弱点是文本替换规则和历史兼容性会带来大量陷阱。学 Shell 不只是学命令，而是在学习如何把 UNIX 的系统调用组织成可组合的工作流。

6 总结与延伸

这节课的主线可以压缩成一句话：终端和 Shell 是 UNIX 把“字符设备”“进程”“文件描述符”“信号”“文本流”组合起来的人机界面。它们看起来老旧，却正因为接口极简而生命力极强。

6.1 核心机制回顾

- **终端的第一性原理**：应用和终端交换字节流；所有复杂行为都来自约定、驱动配置和软件解释。
- **TTY line discipline**：决定输入是 raw 还是 canonical，是否 echo，Ctrl-C/Ctrl-D 如何解释。
- **PTY**：把真实终端抽象成可动态分配的 master/slave 对，支撑终端模拟器、SSH、tmux、ttyrec。
- **Job control**：通过 session、process group、foreground process group 解决“当前任务是谁”的问题。
- **Shell**：把命令文本翻译成系统调用，把小工具用文件描述符和文本流连接起来。

6.2 从历史包袱到设计启发

课堂并没有把 UNIX 的历史设计神圣化。相反，讲者多次强调：许多机制是 1970 年代资源、硬件和工程条件下的妥协。它们被 POSIX 标准固定下来，所以今天仍然要兼容；但如果重新设计系统，也许可以用更现代的任务容器、权限边界、图形会话模型或应用沙箱替代 session/process group 这类混合概念。

学习方式建议

不要把 Shell、TTY、job control 当成孤立知识点背诵。更好的方法是追问每个现象背后的所有权：这个字符由谁解释？这个 fd 在谁手里？这个信号发给哪个集合？这个权限属于 Shell 还是被执行程序？这些问题能把终端现象还原成操作系统机制。

6.3 下一步可以做什么

- 用 `stty -a` 观察当前终端的特殊字符配置，尝试修改 `interrupt` 或 `echo`。

- 写一个 raw mode 小程序，打印每个输入字节的十六进制值。
- 用 `openpty` 或 `/dev/ptmx` 写一个最小 miniTTY，把子进程接到 PTY slave。
- 写一个只支持 `fork/exec/wait`、重定向和管道的 mini shell。
- 阅读 `man sh`、`man termios`、`man setsid`、`man setpgid`，把手册内容映射回本节课的概念图。

6.4 本章小结

终端和 Shell 的价值不在于怀旧，而在于它们展示了操作系统最耐用的设计风格：用小接口承载大世界。一个字节流接口，可以演化出登录、远程连接、复杂 TUI、图像协议和作业控制；一门文本替换语言，可以把文件、进程、管道和权限组织成日常开发 workflow。理解这套机制，等于理解了 UNIX 世界中“简单接口如何组合成复杂系统”的核心范式。